

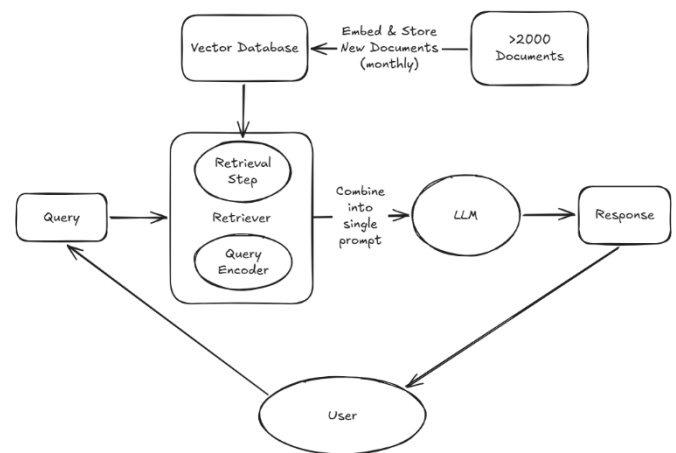
System Design

Architecture

The architecture is that of a traditional retrieval-augmented generation (RAG) pipeline. The key components include a retriever, along with a vector database, as well as the generator (LLM).

The retriever uses a text embedding model, such as OpenAI's text-embedding-3-small, to embed the user query as a vector. The retriever then uses this vector to perform a similarity search with the vector database that stores the vector embeddings of the full library of 2000 documents in order to retrieve the relevant document chunks. This vector database is updated monthly when the library of documents is updated. The retrieved document chunks is then combined with the original query into a single prompt before being fed into the generator.

The generator is an LLM that takes the augmented query as an input and generates a response to it, which is then return to the user.



Key decisions and tradeoffs

Choice of models

Since the design brief requires the response time to feel “snappy”, we assume that to mean that the response time needs to be fast. Given that, for text embedding, it makes sense to choose a smaller model like text-embedding-3-small as opposed to text-embedding-3-large. For the generator LLM, we choose a model that is either not too large or has been quantized.

Vector index vs vector database

Vector database was chosen as it includes additional functionality over vector index, such as data management, metadata association, scalability, and ability to handle real-time updates, which is important as we want to update the database monthly.

Hosting

We assume that the internal documents have some level of security classification. As such, we choose to host the system on-premises instead of on the cloud.

Monitoring

Post-deployment, we would monitor server load, response times, as well as run tests to ensure that the vector database is up to date. We would probably also monitor retrieval and response quality to ensure that the retriever and vector database are functioning correctly.

Possible failure mode

One possible failure mode that would only surface in production is the saturation of the thread pool when the number of concurrent users spikes to above 20, which would bottleneck the system.